Zenith: learning executable observation models

Will Cook • 14 October 2026

Abstract. Zenith retains executable models, measured observations and counterexamples. A language model proposes code; the host executes it and returns actual failures. We test whether changing a program's interfaces or computation makes later predictions more reliable. A small structural-revision experiment repairs information loss and rollback-sensitive state, but ordinary code with function contracts outperforms explicit local graph context. A harder synthetic study separates source-rule induction from code-delivery failures and compares cheap models with reasoning disabled or enabled. Earlier native SQLite studies favor retaining executable alternatives and evidence, but unchanged early code ties the initial final learner. These bounded results guide an implementation; they do not establish graph superiority, compact-state discovery or recursive self-improvement.

1. Retain a model that observations can challenge. Let $D_s = \{(a_i, o_i)\}$ be host-recorded experiments on source s. An executable program P_s uses the evidence to predict a requested quantity q after a new action sequence a:

$$P_s(a, q, D_s) \longmapsto (v, u), \qquad u = 1 \implies v = \text{null}.$$

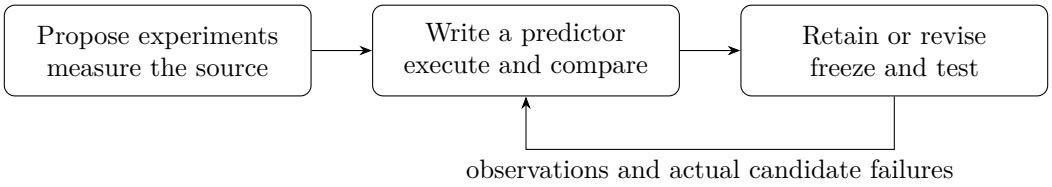
The retained object is code, source binding, measured receipts and counterexamples. The model's scope statement remains a hypothesis. A hash binds bytes; it does not show that the source's meaning is unchanged. Identifiability is specific to the question: uncertainty about one counter need not prevent a supported conclusion about committed state. After a model is refuted, however, partial reuse needs a justified component contract; surviving marginal fits alone do not establish a valid joint model.

A source whose numbers have different meanings. Each experiment starts with an empty SQLite table and two writing connections. The public actions are insertion, update, deletion and transaction control. The hidden source configuration varies conflict handling, an insertion trigger and counter selection. A separate connection measures committed count and value sum. The emitted counter can count direct changes, connection-total changes, or the totals across both writers. These are actual SQLite executions, not synthetic labels produced by a model. Native counters differ in whether they include trigger effects or another connection's writes [4].

For example, two insertions of the same key can preserve, replace or update a value. Rolling back can remove visible effects while a change counter retains work already performed. A no-op update can increment a counter without changing the stored value. Past measurements must distinguish these possibilities before a later compound sequence can be answered reliably. The model receives action documentation, the allowed source family and calibration receipts. Actual settings, candidate programs and the source implementation stay private.

The study supplies three useful quantities: channel, committed count and sum. It does not claim that the model invented those goals. The private evaluator executes every compatible configuration in a frozen 18-member family. A query is supported if their values agree. This identification condition is relative to that family, not a guarantee against every source consistent with finite data.

2. One investigation loop. The model proposes discriminating experiments. The host records their actual outcomes and an independent challenge. It then executes a proposed predictor on every observed development case. The next request receives the selected program and the actual last challenger, including rejected code and measured execution failures. Search can also restart from the same measurements without the failed code. Evidence persists across replacement; a candidate is retained only under the declared development rule. Lab, Oracle and Evolve name functions of this loop, not mandatory separate model agents.



Initial and repair trajectories retain the latest executable candidate whose development score is at least the incumbent's. Ties permit revision without claiming improvement; regressions remain recorded. A lookup can fit development, so further-case evaluation is essential. The initial six-source comparison matches observation history, tools and six proposals per source across incremental, archive and fresh construction. An unchanged early program with final history tests whether a stable interpreter already explains the benefit.

Structural revision. Four tasks expose a lost calibration count, lost SQLite allocation history, lost event keys/times, and a correct-program control. Global ordinary coding, local caller/consumer context and ordinary function contracts all permit whole program reconstruction, changed interfaces and new state. A fixed-interface condition is diagnostic. Each receives the same eight calibration and 24 other development cases, two proposals, and its actual execution feedback. Two separate 24-case panels per task change legal action structures. The graph is the call structure of ordinary code, not a special execution engine.

Source-rule induction. Three synthetic integer-sequence sources belong to a supplied 4,464-member family: a small polynomial, optional adjacent-value or preceding-sum term, and one to four cumulative-sum passes. Twelve observations identify each selected source within that family; 24 more provide development feedback. Coefficients, depth and interaction stay private. Later histories have lengths 8–14 and 16–32, versus at most six in development. The model receives whole histories, so success need not use compact persistent state. Global, contextual and contract prompts are compared with Qwen and DeepSeek. A later comparison uses six fresh configurations, repaired syntax delivery and DeepSeek reasoning off versus low; a global ordinary-code condition tests the need for contracts.

Budget and custody. Current inexpensive Chinese-developed models use explicit pinned endpoints, at most three concurrent calls across the host, and one per endpoint. Reasoning and output limits are frozen per study. Low reasoning has a larger allowance; this is not an equal-token causal comparison. Syntax failures remain outcomes. A changed executor begins a new study; archived source preserves old failures. Earlier native studies also compare fresh compilation, retained candidate pools and model-proposed or fixed source experiments at stated budgets.

All construction is frozen before protected scoring; failed outputs remain unchanged. Sources are configured members of one disclosed software family, not independent domains. Different screens are not pooled into one accuracy.

3. Evidence and limits. Seven new studies made 122 calls for $0.177965 reported inference. Each study preserves all original proposals, individual answers and archived runtime. The table gives selected confirmation scores; screening totals match except for initial/fixed structural controls, which are reported in the repository.

Study / setting	Global	Context	Contracts	Cost ($)
Qwen, structural	88/96	88/96	96/96	.045607
Qwen, rules, off	0/72	24/72	0/72	.019720
DeepSeek, rules, off	0/72	0/72	0/72	.014219
DeepSeek, rules, low	48/72	48/72	48/72	.035843
Fresh bindings, off	—	—	52/144	.009421
Fresh bindings, low	144/144	—	120/144	.053155

The last row combines costs of two studies sharing one dataset. Different panels are not pooled; individual answers are not independent source replications.

A real repair, and an unobserved regression. The structural contracts trajectory corrects rollback-sensitive SQLite allocation state: its first and second proposals score 14/24 and 24/24 on confirmation. Global and contextual programs each pass all 32 visible cases, yet score 16/24 later because they omit pending inserts when measurement occurs inside an open transaction. Such transactions are legal but absent from visible examples. Ordinary code also repairs lost sensor multiplicity and event keys. The graph-context condition earns no advantage; the correct-code control stays correct in every condition. Fixed-interface code confirms at 32/96, including invalid proposals retained as failures.

Acquire rules without calling compilation self-improvement. The first rule studies expose both invented parameters and rejected syntax. The old executor rejects even the harmless single _ loop variable. A narrow correction and explicit syntax examples apply to every fresh-binding condition; archived replay preserves the old failures. With that interface, DeepSeek low reasoning and global coding acquire all six source rules on their *first* proposal. Second proposals add no measured benefit. Some successful programs retain full lists or refit embedded observations per query. These results establish whole-history rule acquisition. An authored exhaustive fitter also passes 144/144 without model calls; the supplied family makes this possible.

Earlier native evidence. An initial 126-call comparison scored 244/288, tied by unchanged early code. Later candidate-pool and larger-archive studies reached 279/288 and 286/288 on different panels and budgets. Five composition attempts stopped before scoring. These favor retained evidence and alternatives, not graph superiority.

Interpretation. Use the simplest successful organisation. Next require a bounded persistent record, checking every streaming prefix and forked continuation; then test whether a model-derived investigation rule reduces acquisition cost on fresh sources against a competent fixed policy. Neither is demonstrated here. PAL [1], Voyager [2] and DreamCoder [3] supply relevant prior art. A feedback loop alone is not an improved learning method.

References

[1] L. Gao et al. *PAL: Program-aided Language Models*. 2022. arXiv:2211.10435.

[2] G. Wang et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. 2023. arXiv:2305.16291.

[3] K. Ellis et al. *DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning*. 2020. arXiv:2006.08381.

[4] SQLite. *Count the Number of Rows Modified; Total Number of Rows Modified*. changes; total_changes. Accessed 13 October 2026.